



Redes Neurais Recorrentes

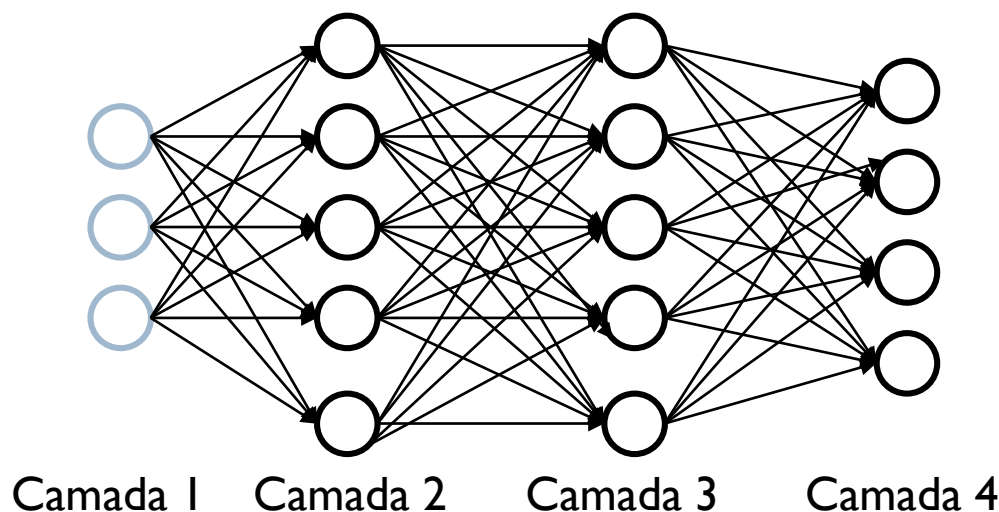
Profs. Bruno Nogueira e Edson Takashi

Agradecimentos

- ▶ Algumas figuras utilizadas nestes slides foram retiradas, com permissão do autor, do livro: C. Aggarwal. Neural Networks and Deep Learning, Springer, 2018.

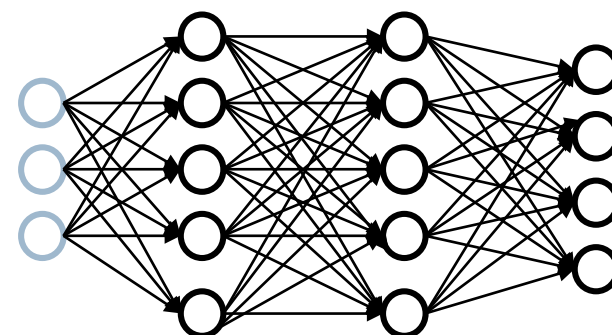
Contextualizando

- ▶ Redes neurais que falamos até agora, de maneira geral, eram redes *feedforward*
 - ▶ Não há ciclos nas conexões de neurônios
 - ▶ Neurônios de uma camada são ligados aos neurônios da camada subsequente



Contextualizando

- ▶ Arquiteturas discutidas são específicas para dados multidimensionais
 - ▶ Atributos não têm grande dependência
 - ▶ Número de dimensões e saídas é fixo
 - ▶ Essa assumption, entretanto, não é válida para uma série de problemas: texto, dados biológicos, séries temporais...
- ▶ Redes neurais recorrentes servem para lidar com este problema



Camada 1 Camada 2 Camada 3 Camada 4

Classificação de textos

- ▶ Abordagens tradicionais consideram o um texto como uma *bag-of-words*
 - ▶ Tabela atributo-valor
 - ▶ Linhas: documentos
 - ▶ Colunas: atributos (palavras)

	exponent	embed	impos	summar	car
Doc1	14	0	10	0	0
Doc2	0	5	0	0	3
Doc3	0	0	0	0	0
Doc4	0	0	0	1	0
Doc5	2	0	0	1	1

Classificação de textos

- ▶ **Abordagem BOW ignora a ordem das palavras**
 - ▶ Não funciona com aplicações que demandam interpretação das sentenças
 - ▶ Não adequada para textos pequenos
- ▶ **Exemplos**

D1: O gato perseguiu o rato.

D2: O rato perseguiu o gato.

	gato	rato	perseguiu
Doc1	1	1	1
Doc2	1	1	1

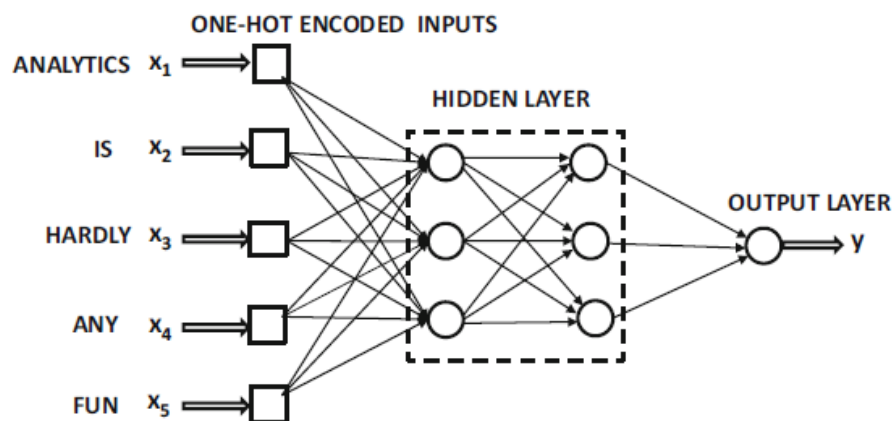
Classificação de textos

- ▶ **Desafio é criar uma arquitetura com um tamanho de entrada variável e um número fixo de parâmetros**
 - ▶ Alguns casos, até mesmo o número de saídas pode ser variável
 - ▶ Exemplo: Tradução automática
- ▶ **Pequenas mudanças na ordem das palavras pode levar a conotações semanticamente diferentes**
 - ▶ Importante codificar, de alguma forma, a ordem das palavras na arquitetura da rede
 - ▶ Reduzir o número de parâmetros necessários com o aumento do tamanho das sequências

Classificação de textos

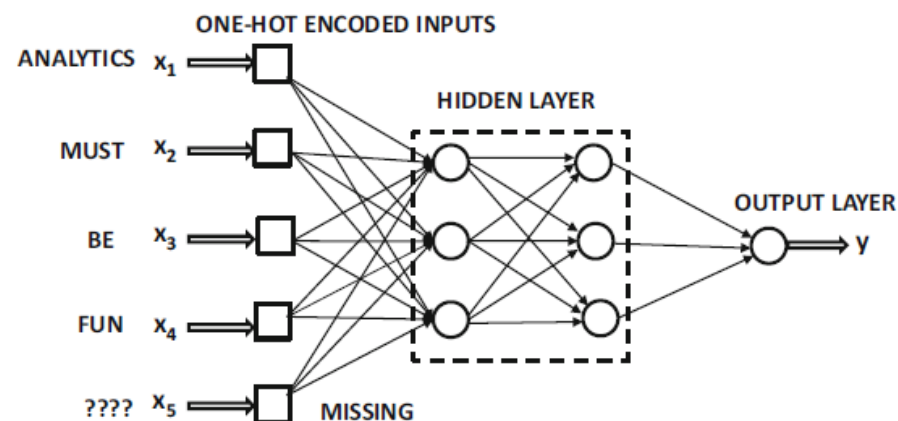
► One hot encoding

► Problema das entradas de tamanho variável



(a) 5-word sentence

"Analytics is hardly any fun."



(b) 4-word sentence

"Analytics must be fun."

Processamento de sequências

- ▶ De maneira geral, o processamento de qualquer sequência requer:
 1. A habilidade de receber e processar entradas na mesma ordem em que aparecem na sequência
 2. Tratar as entradas a cada intervalo de tempo de maneira similar em relação às entradas anteriores
- ▶ Número fixo de parâmetros + processar entradas de tamanho variável
- ▶ Redes neurais recorrentes atendem a essas demandas

Redes neurais recorrentes

Recurrent Neural Networks (RNN)

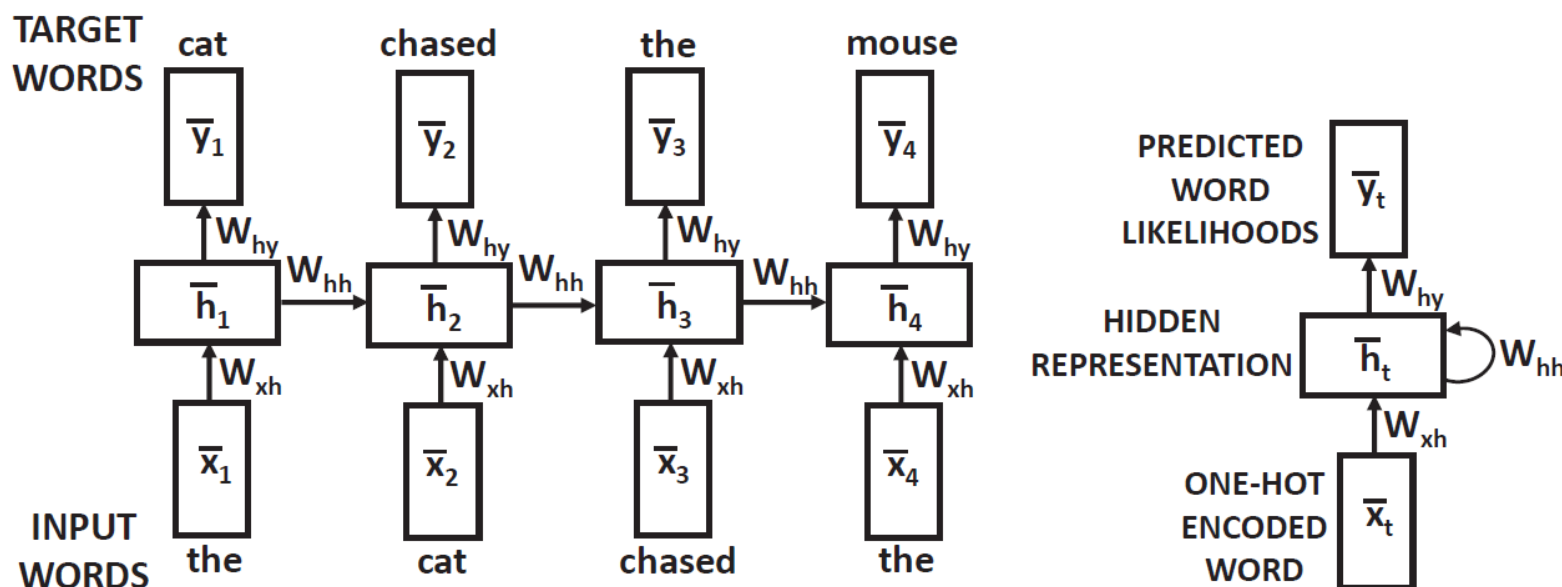
- ▶ Há relação um-para-um entre as camadas da rede neural e as posições específicas em uma sequência (*time stamps*)
 - ▶ Ao invés de um número de entradas variável em um única camada de entrada, redes recorrentes têm um número variável de camadas – cada camada tem uma única entrada associada a uma posição na sequência (um *time stamp*)
 - ▶ Cada camada se comunica com as camadas posteriores
 - ▶ Cada camada utiliza o mesmo conjunto de parâmetros
 - ▶ Garante modelagem idêntica, independente da posição na sequência
 - ▶ Número de parâmetros é fixo
- ▶ A mesma arquitetura repete-se ao longo do tempo - recorrente

Redes neurais recorrentes

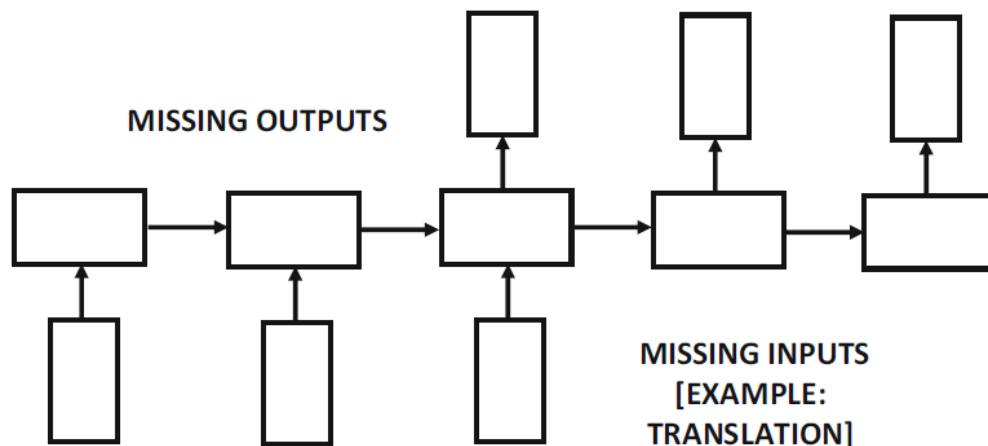
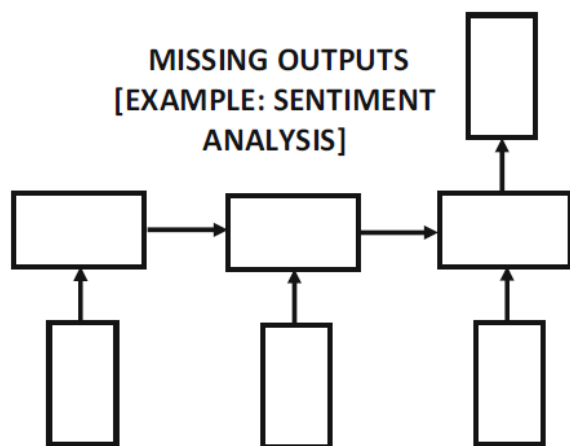
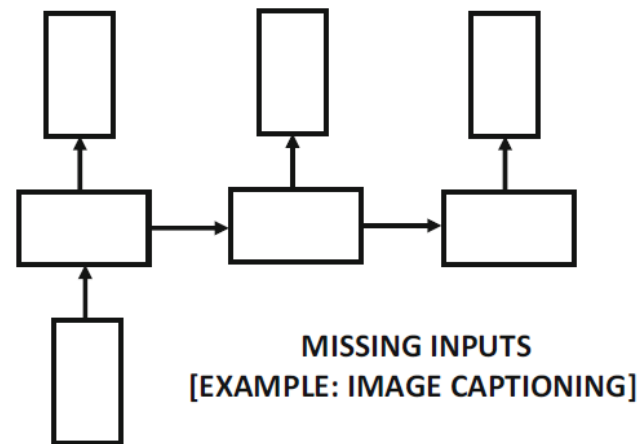
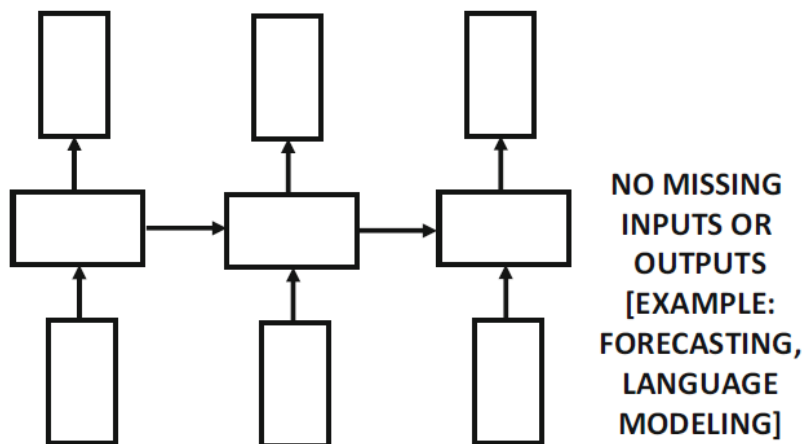
- ▶ São redes feed-forward com estrutura específica baseada na noção de *time layering* – construção de camadas temporais
 - ▶ Entrada é uma sequência de valores
 - ▶ Gera uma sequência de saídas
 - ▶ O i -ésimo elemento da sequência é apresentado à i -ésima camada
 - ▶ **Propriedade de Markov:** qualquer elemento de uma sequência pode ser predito a partir dos elementos antecessores utilizando um modelo fixo

Arquitetura de uma RNN

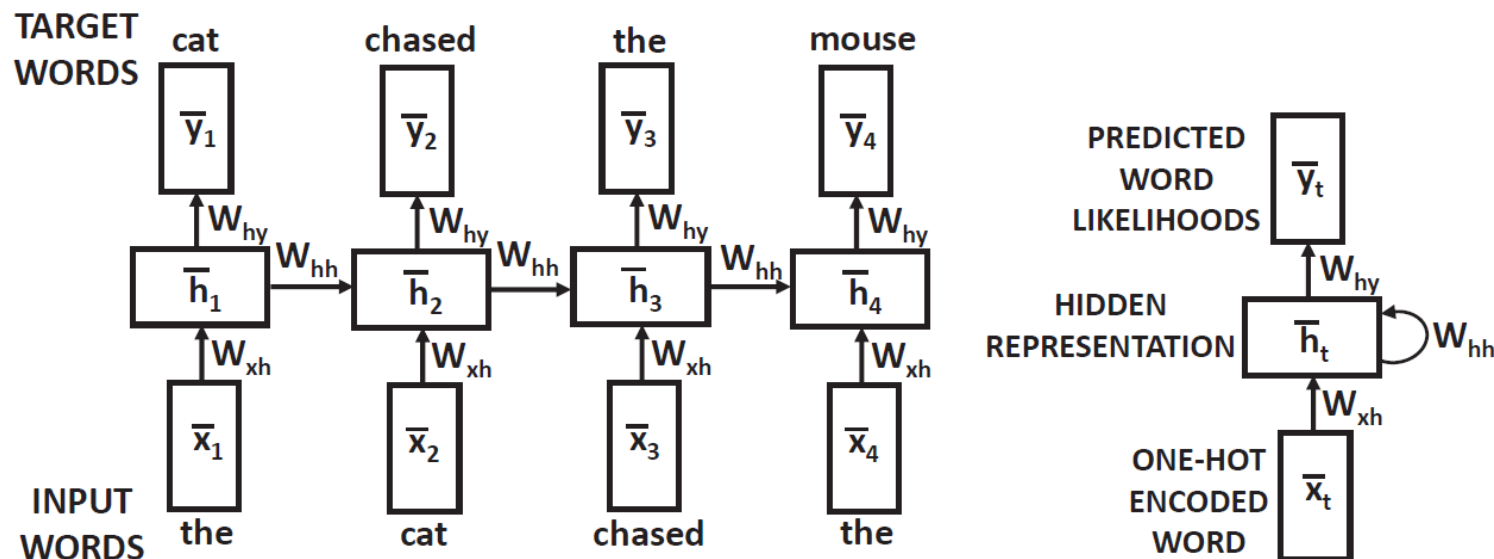
- ▶ Vamos utilizar como exemplo uma RNN em nível de palavra
 - ▶ Modelo de linguagem
 - ▶ Predição da próxima palavra na sequência (vetor de probabilidades, do tamanho do vocabulário)



Arquitetura de uma RNN

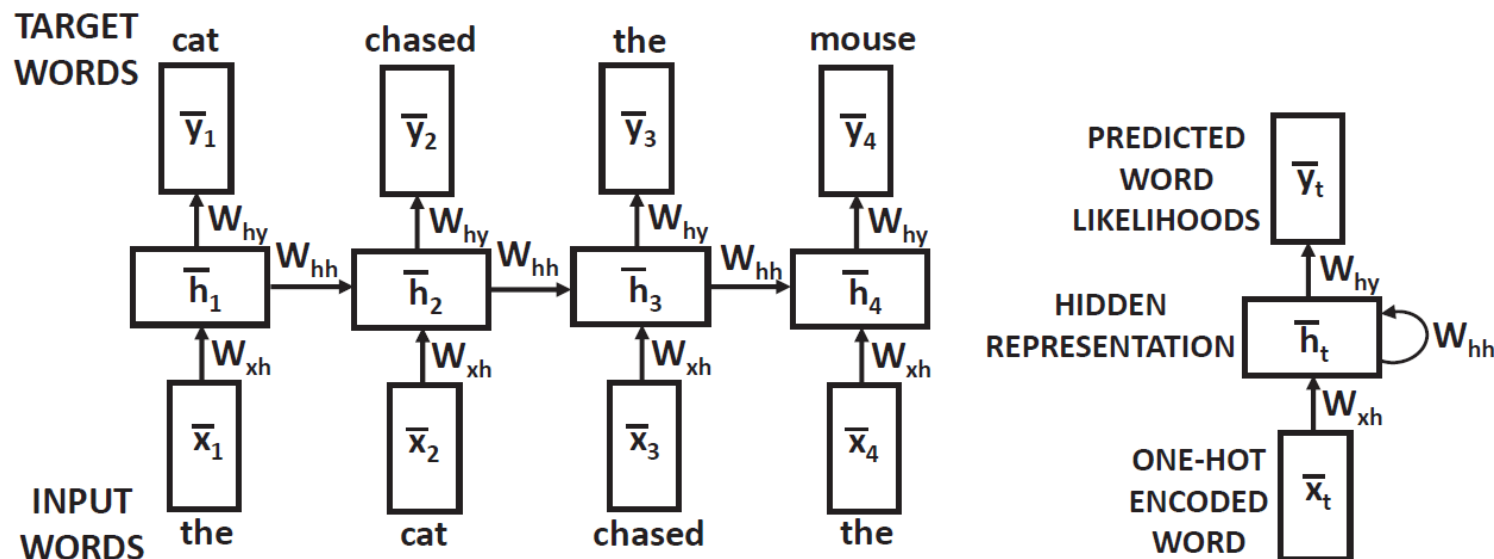


Arquitetura de uma RNN



$$\begin{aligned} \bar{h}_t &= f(\bar{h}_{t-1}, \bar{x}_t) \\ \bar{y}_t &= g(\bar{h}_t) \end{aligned} \quad \longrightarrow \quad \begin{aligned} \bar{h}_t &= \tanh(W_{hh}\bar{h}_{t-1} + W_{xh}\bar{x}_t) \\ \bar{y}_t &= W_{hy}\bar{h}_t \end{aligned}$$

Arquitetura de uma RNN



Vetor de tamanho p (tamanho da representação densa)

$$\bar{h}_t = f(\bar{h}_{t-1}, \bar{x}_t)$$

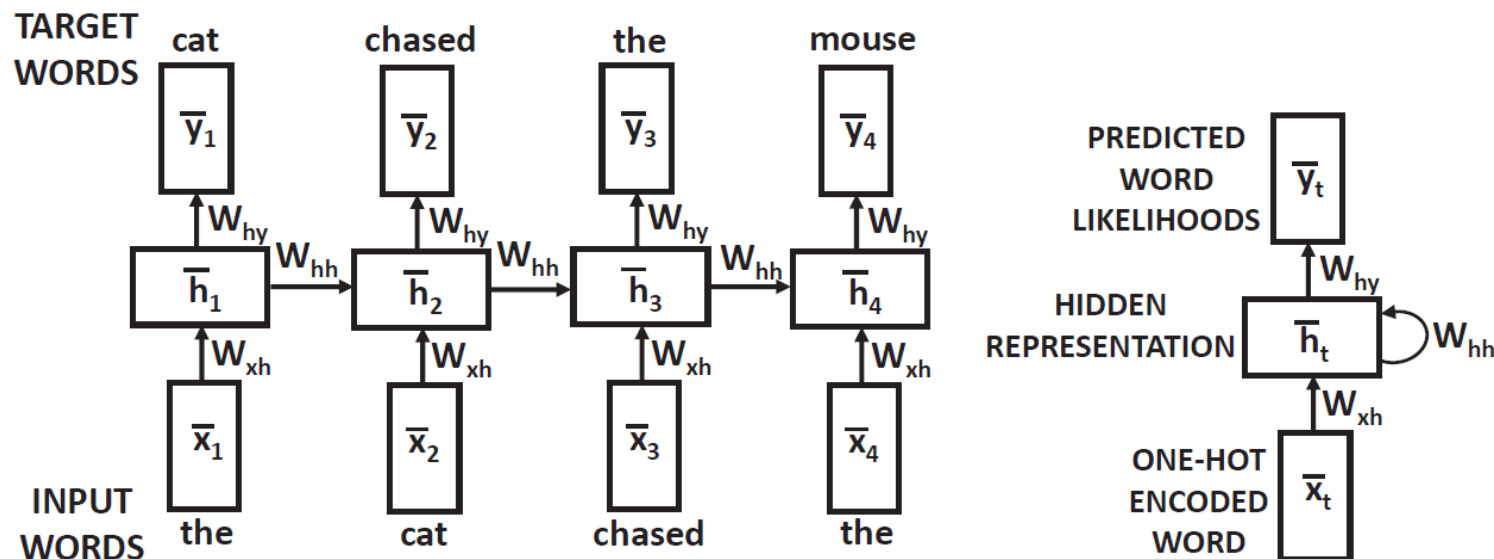
$$\bar{y}_t = g(\bar{h}_t)$$

$$\bar{h}_t = \tanh(W_{hh}\bar{h}_{t-1} + W_{xh}\bar{x}_t)$$

$$\bar{y}_t = W_{hy}\bar{h}_t$$

Vetores de tamanho d (tamanho do vocabulário)

Arquitetura de uma RNN

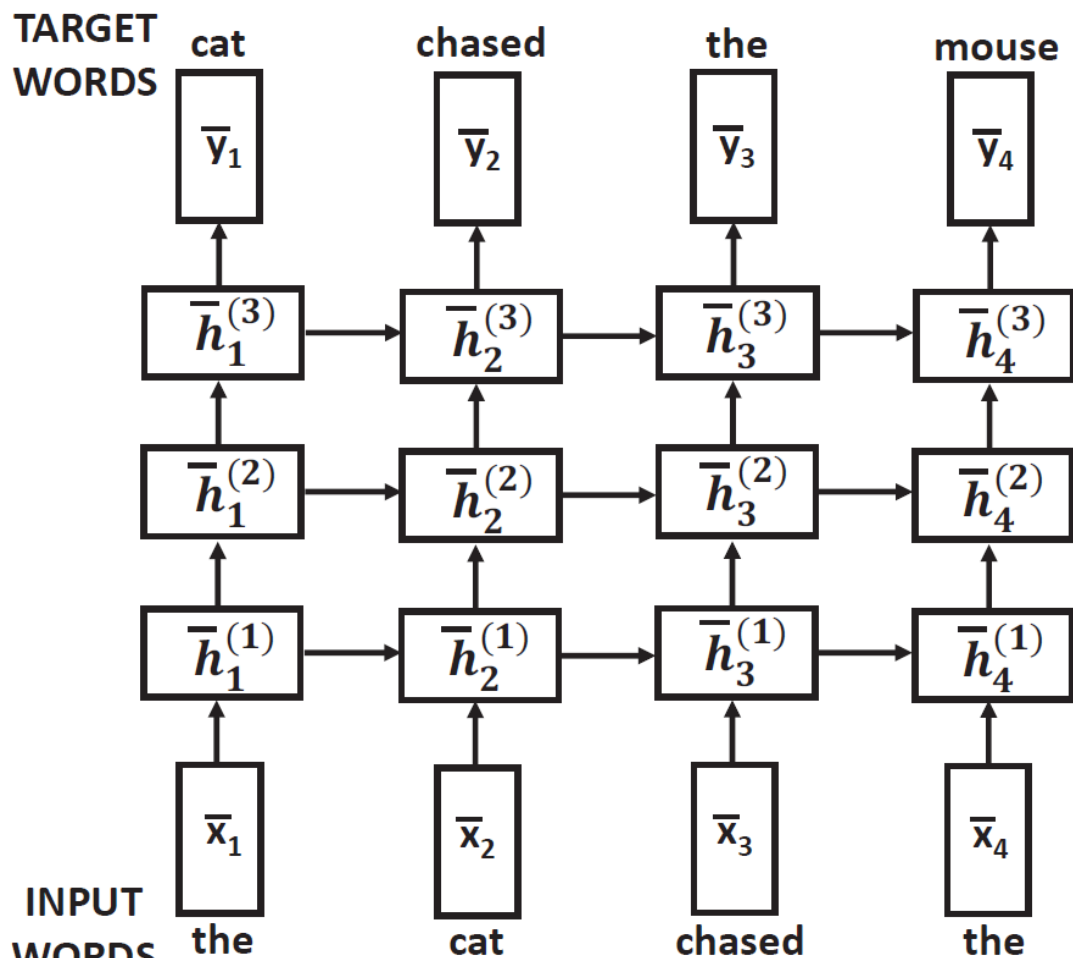


$$\begin{aligned} \bar{h}_t &= f(\bar{h}_{t-1}, \bar{x}_t) \\ \bar{y}_t &= g(\bar{h}_t) \end{aligned} \quad \Rightarrow \quad \begin{aligned} \bar{h}_t &= \tanh(W_{hh}\bar{h}_{t-1} + W_{xh}\bar{x}_t) \\ \bar{y}_t &= W_{hy}\bar{h}_t \end{aligned}$$

Matriz $p \times p$

Matriz $p \times d$ - Cada d -ésima coluna armazena o *embedding* de uma palavra do vocabulário

Arquitetura de uma RNN multicamadas



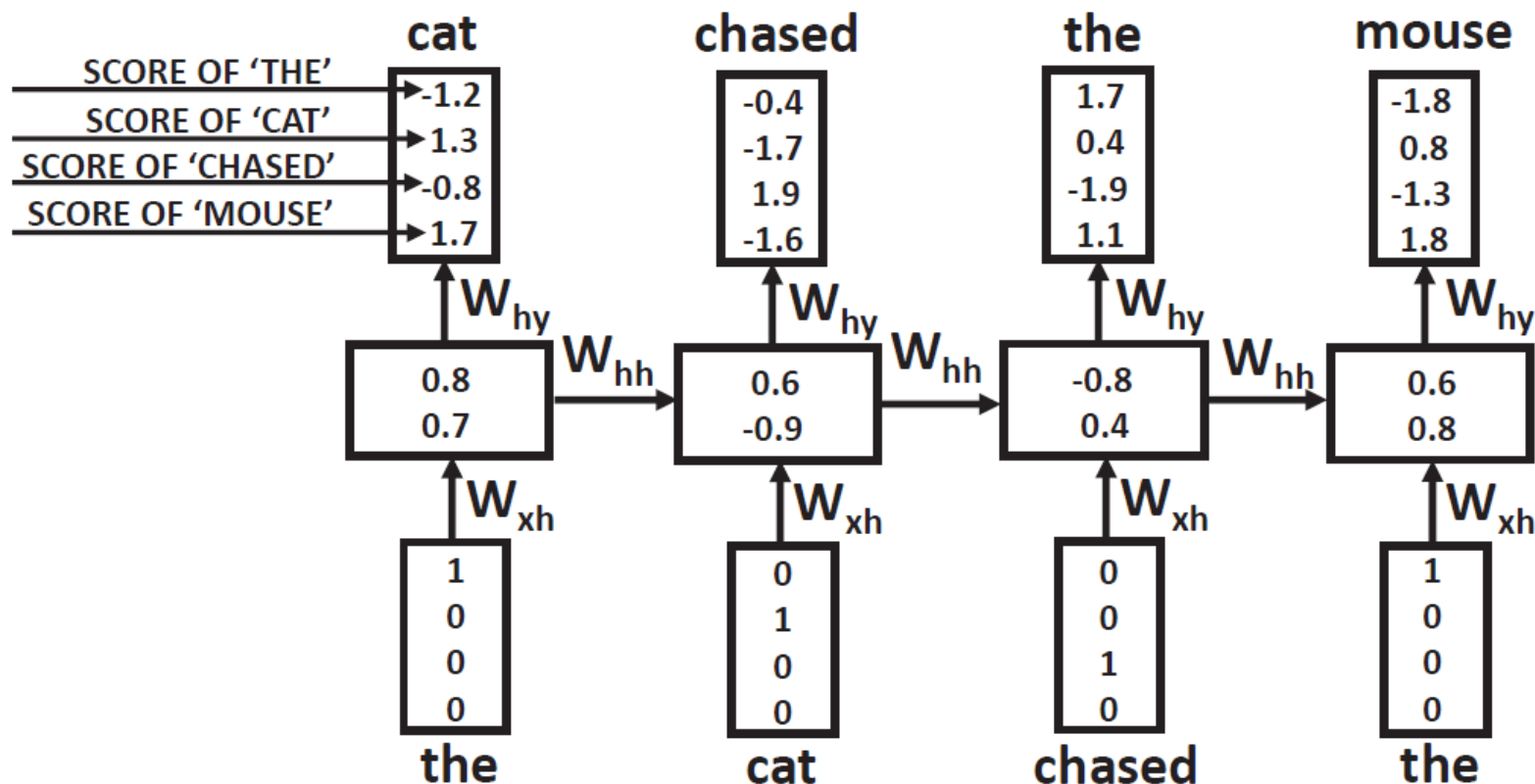
$$W = [W_{xh}, W_{hh}]$$

$$\bar{h}_t^{(k)} = \tanh W^{(k)} \begin{bmatrix} \bar{h}_t^{(k-1)} \\ \bar{h}_{t-1}^{(k)} \end{bmatrix}$$

Entradas de tamanhos distintos

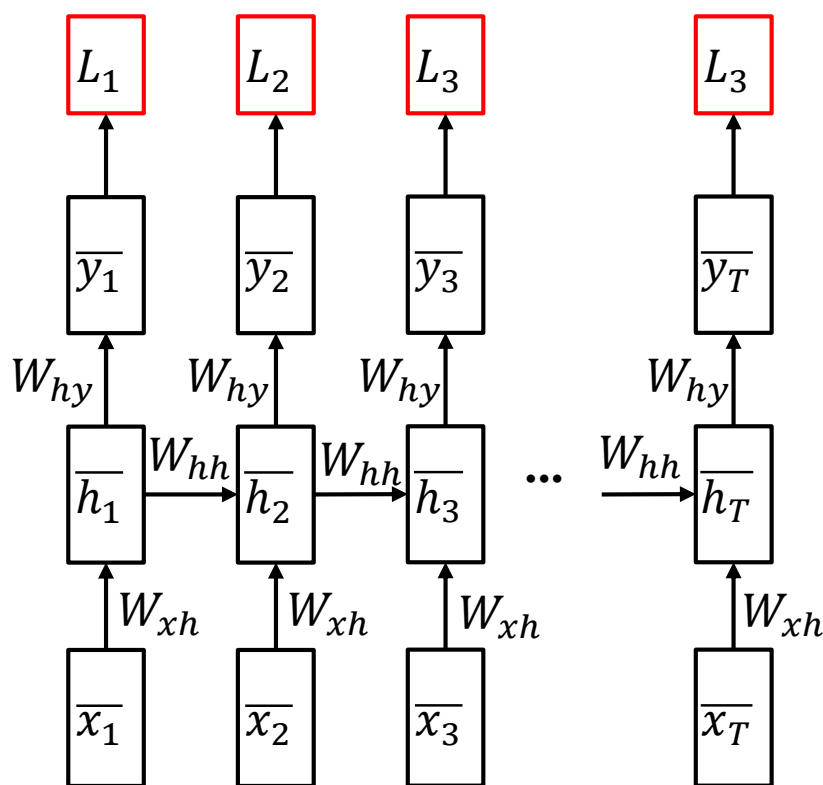
- ▶ A relação de recorrência $\bar{h}_t = f(\bar{h}_{t-1}, \bar{x}_t)$ provê à RNN a habilidade de computar uma função de entradas de tamanho variável
 - ▶ $\bar{h}_t$ é calculado em termos de t entradas
 - ▶ $\bar{h}_1 = f(\bar{h}_0, \bar{x}_1)$ e $\bar{h}_2 = f(f(\bar{h}_0, \bar{x}_1), \bar{x}_2)$
 - ▶ $\bar{h}_t$ é uma função de $\bar{x}_1, \bar{x}_2, \dots, \bar{x}_t$
 - ▶ Saída baseada em toda a sequência de entrada

Exemplo: Modelos de linguagem



Treinamento de uma RNN

- Modificação do algoritmo *backpropagation* pode ser utilizada para treinar RNN
 - *Backpropagation through time*

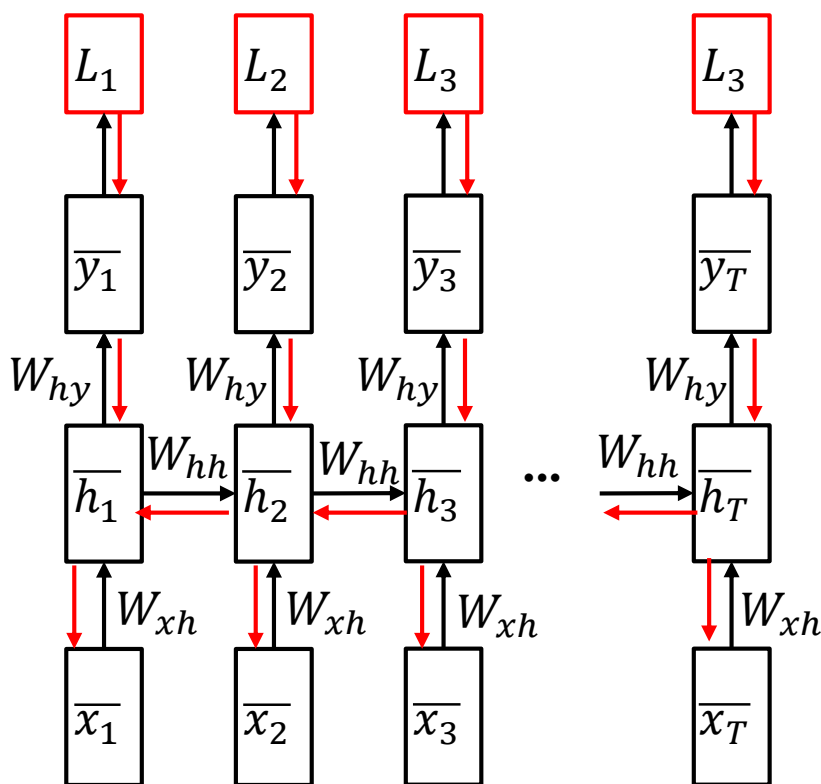


$$L = \sum_{i=1}^T L_i$$

Fase *forward*

Treinamento de uma RNN

- Modificação do algoritmo *backpropagation* pode ser utilizada para treinar RNN
 - *Backpropagation through time*



$$L = \sum_{i=1}^T L_i$$

Fase backward

Backpropagation through time

1. Computa a *loss* para cada posição da sequência
2. Assumindo que os pesos $W_{xh}^{(t)}$, $W_{hh}^{(t)}$ e $W_{hy}^{(t)}$ são diferentes para diferentes posições t , pode-se utilizar o *backpropagation* convencional para computar $\frac{\partial L}{\partial W_{xh}^{(t)}}$, $\frac{\partial L}{\partial W_{hh}^{(t)}}$

$$\text{e } \frac{\partial L}{\partial W_{hy}^{(t)}}$$

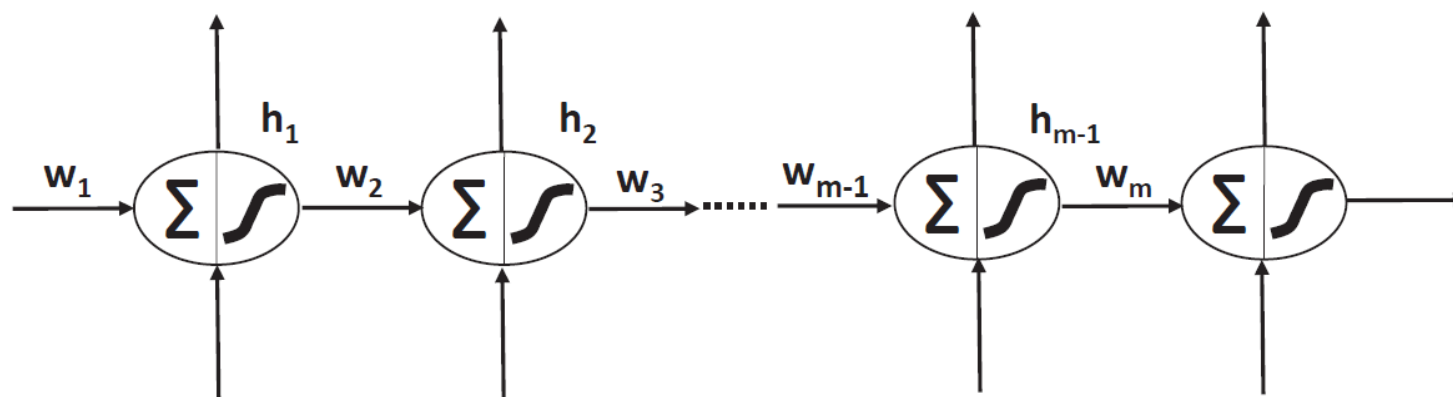
3. Soma-se os gradientes

$$\frac{\partial L}{\partial W_{xh}} = \sum_{t=1}^T \frac{\partial L}{\partial W_{xh}^{(t)}} \quad \frac{\partial L}{\partial W_{hh}} = \sum_{t=1}^T \frac{\partial L}{\partial W_{hh}^{(t)}} \quad \frac{\partial L}{\partial W_{yh}} = \sum_{t=1}^T \frac{\partial L}{\partial W_{yh}^{(t)}}$$

4. Atualiza-se os pesos utilizando gradiente descendente (ou algum outro algoritmo de otimização adequado)

Explosão / dissipação de gradiente

- ▶ Um problema comum no treinamento de RNN é a dissipação ou explosão de gradiente



- ▶ A derivada parcial retropropagada é multiplicada por pesos e derivadas de funções de ativação
- ▶ As derivadas parciais vão continuamente aumentar (explodir) ou diminuir (dissipar)

Explosão / dissipação do gradiente

- ▶ Já vimos algumas formas de amenizar este problema
 - ▶ Inicialização dos vetores
 - ▶ Regularização dos parâmetros
 - ▶ *Batch norm*
 - ▶ *Gradient clipping*
- ▶ Não evitam, no entanto, o problema
 - ▶ Redes recorrentes são “muito profundas”
 - ▶ Depende do tamanho da sequência a ser considerada

R. Pascanu, T. Mikolov, and Y. Bengio. 2013. *On the difficulty of training recurrent neural networks*. In Proceedings of the 30th International Conference on International Conference on Machine Learning (ICML'13), Sanjoy Dasgupta and David McAllester (Eds.), Vol. 28. JMLR.org III-1310-III-1318.

Long Short-Term Memory (LSTM)

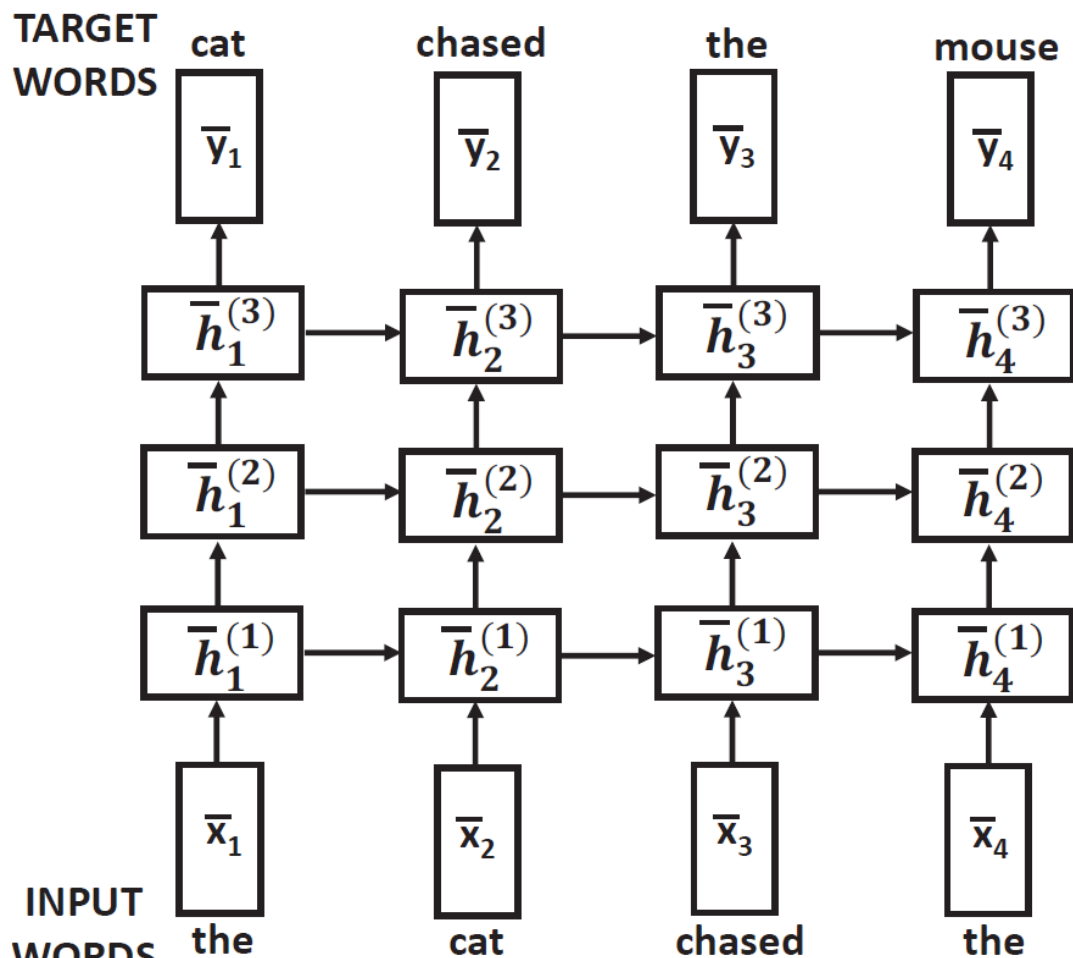
- ▶ Uma rede neural que utiliza atualizações multiplicativas é boa apenas no aprendizado sob sequências curtas
 - ▶ Boa memória de curto prazo
 - ▶ Memória de longo prazo ruim
- ▶ Solução: modificar a equação de recorrência do vetor oculto com o uso de memória de longo prazo (LSTM)
 - ▶ Uma rede neural recorrente carrega apenas um estado escondido $\bar{h}_t^{(k)}$ ao longo das camadas temporais
 - ▶ Uma LSTM carrega tanto $\bar{h}_t^{(k)}$ (memória de curto prazo) quanto um estado célula $\bar{c}_t^{(k)}$ (memória de longo prazo)

S. Hochreiter, Y. Bengio, P. Frasconi, and J. Schmidhuber. Gradient flow in recurrent nets: the difficulty of learning long-term dependencies, A Field Guide to Dynamical Recurrent Neural Networks, IEEE Press, 2001.

Long Short-Term Memory (LSTM)

- ▶ **Gates** são utilizados para controlar os updates de camada para camada
 - ▶ Memória de curto prazo $\bar{h}_t^{(k)}$ é atualizada mais agressivamente
 - ▶ Memória de longo prazo $\bar{c}_t^{(k)}$ é atualizada mais devagar
 - ▶ Troca de informações entre memória de longo prazo e memória de curto prazo é permitida
- ▶ Estado célula mantém uma parte da informação dos estados célula anteriores
 - ▶ Modela dependências de longo prazo
 - ▶ Atualização mais lenta implica em maior persistência de informação
 - ▶ Implica, também, em gradientes menores e menores chances de dissipação ou explosão de gradiente

Relembrando...



$$W = [W_{xh}, W_{hh}]$$

$$\bar{h}_t^{(k)} = \tanh W^{(k)} \begin{bmatrix} \bar{h}_t^{(k-1)} \\ \bar{h}_{t-1}^{(k)} \end{bmatrix}$$

Long Short-Term Memory (LSTM)

- Assumindo que o estado escondido e o estado célula são vetores p -dimensionais
- A matriz $W^{(k)}$ é de tamanho $4p \times 2p$
 - $4p \times (p+d)$, quando $k = 1$
- As variáveis intermediárias são vetores p -dimensionais $\bar{i}$, $\bar{f}$, $\bar{o}$ e $\bar{c}$ que podem ser empilhados para criar um vetor de dimensão $4p$

$$\begin{array}{l} \text{Input Gate:} \\ \text{Forget Gate:} \\ \text{Output Gate:} \\ \text{Cell Increment:} \end{array} \begin{bmatrix} \bar{i} \\ \bar{f} \\ \bar{o} \\ \bar{c} \end{bmatrix} = \begin{pmatrix} \text{sigm} \\ \text{sigm} \\ \text{sigm} \\ \text{tanh} \end{pmatrix} W^{(k)} \begin{bmatrix} \bar{h}_t^{(k-1)} \\ \bar{h}_{t-1}^{(k)} \end{bmatrix}$$

Long Short-Term Memory (LSTM)

- Assumindo que o estado escondido e o estado célula são vetores p -dimensionais
- A matriz $W^{(k)}$ é de tamanho $4p \times 2p$
 - $4p \times (p+d)$, quando $k = 1$
- As variáveis intermediárias são vetores p -dimensionais $\bar{i}$, $\bar{f}$, $\bar{o}$ e $\bar{c}$ que podem ser empilhados para criar um vetor de dimensão $4p$

$$\begin{array}{l}
 \text{Input Gate:} \\
 \text{Forget Gate:} \\
 \text{Output Gate:} \\
 \text{Cell Increment:}
 \end{array}
 \begin{bmatrix} \bar{i} \\ \bar{f} \\ \bar{o} \\ \bar{c} \end{bmatrix} = \begin{pmatrix} \text{sigm} \\ \text{sigm} \\ \text{sigm} \\ \text{tanh} \end{pmatrix} W^{(k)} \begin{bmatrix} \bar{h}_t^{(k-1)} \\ \bar{h}_{t-1}^{(k)} \end{bmatrix}$$

Vetores no intervalo $[0,1]$. Na multiplicação, funcionam como “portas lógicas AND”.

Long Short-Term Memory (LSTM)

- ▶ Atualização dos estados célula e estados escondidos
 - ▶ Seletivamente “esqueça” ou adicione para a memória de longo prazo

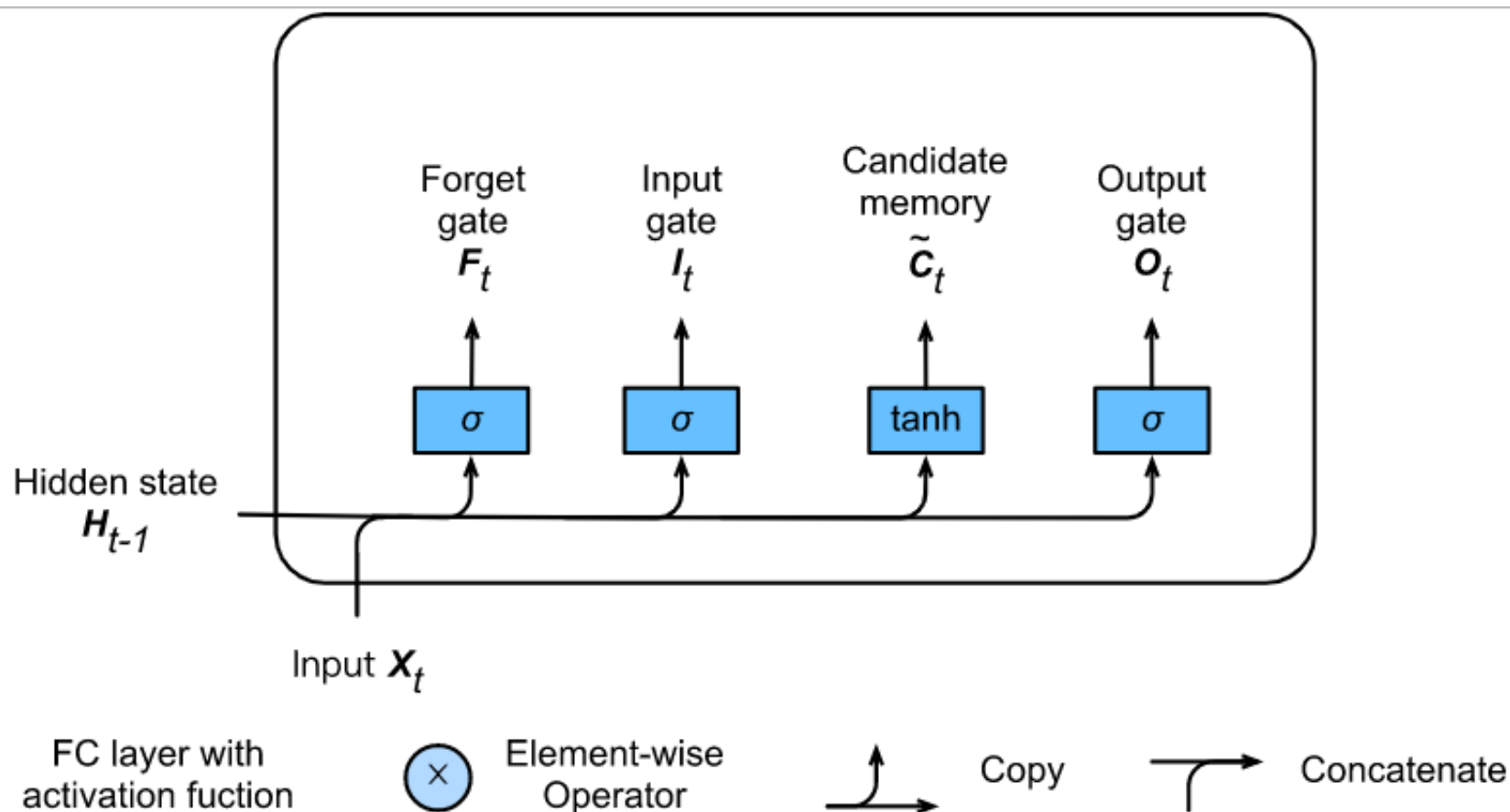
$$\bar{c}_t^{(k)} = \underbrace{\bar{f} \odot \bar{c}_{t-1}^{(k)}}_{\text{Reset?}} + \underbrace{\bar{i} \odot \bar{c}}_{\text{Increment?}}$$

- ▶ “Vazamento” seletivo da memória de longo termo para o estado escondido

$$\bar{h}_t^{(k)} = \bar{o} \odot \tanh(\bar{c}_t^{(k)})$$

Long Short-Term Memory (LSTM)

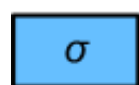
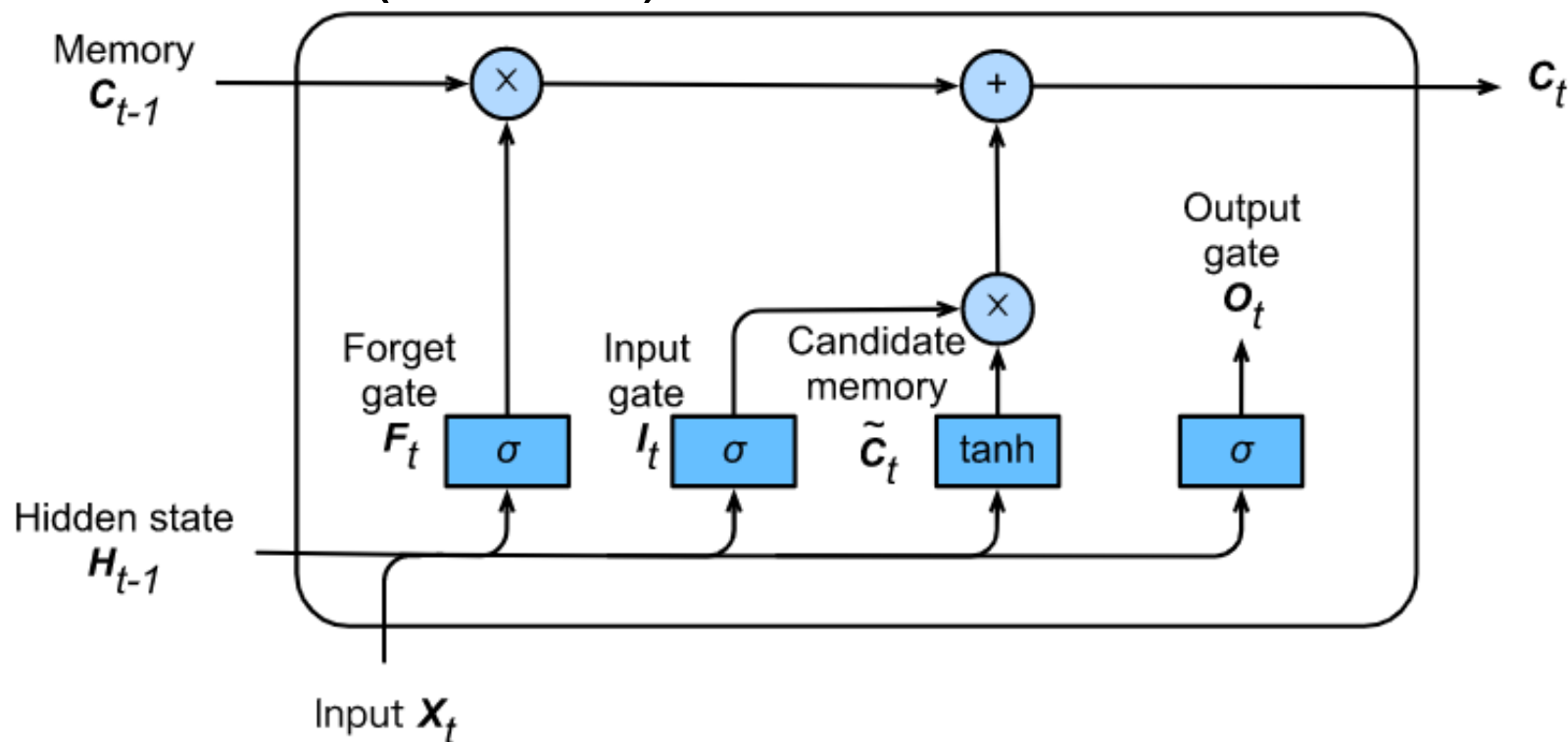
- Cálculo das portas *forget*, *input*, *output* e candidata



Fonte: https://www.d2l.ai/chapter_recurrent-neural-networks/lstm.html

Long Short-Term Memory (LSTM)

► Estado célula (memória)



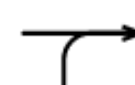
FC layer with
activation function



Element-wise
Operator



Copy

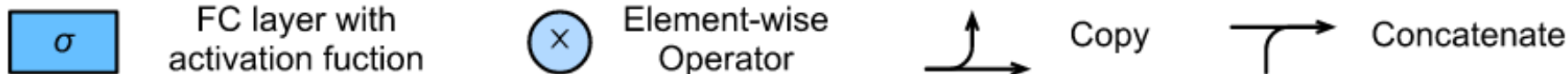
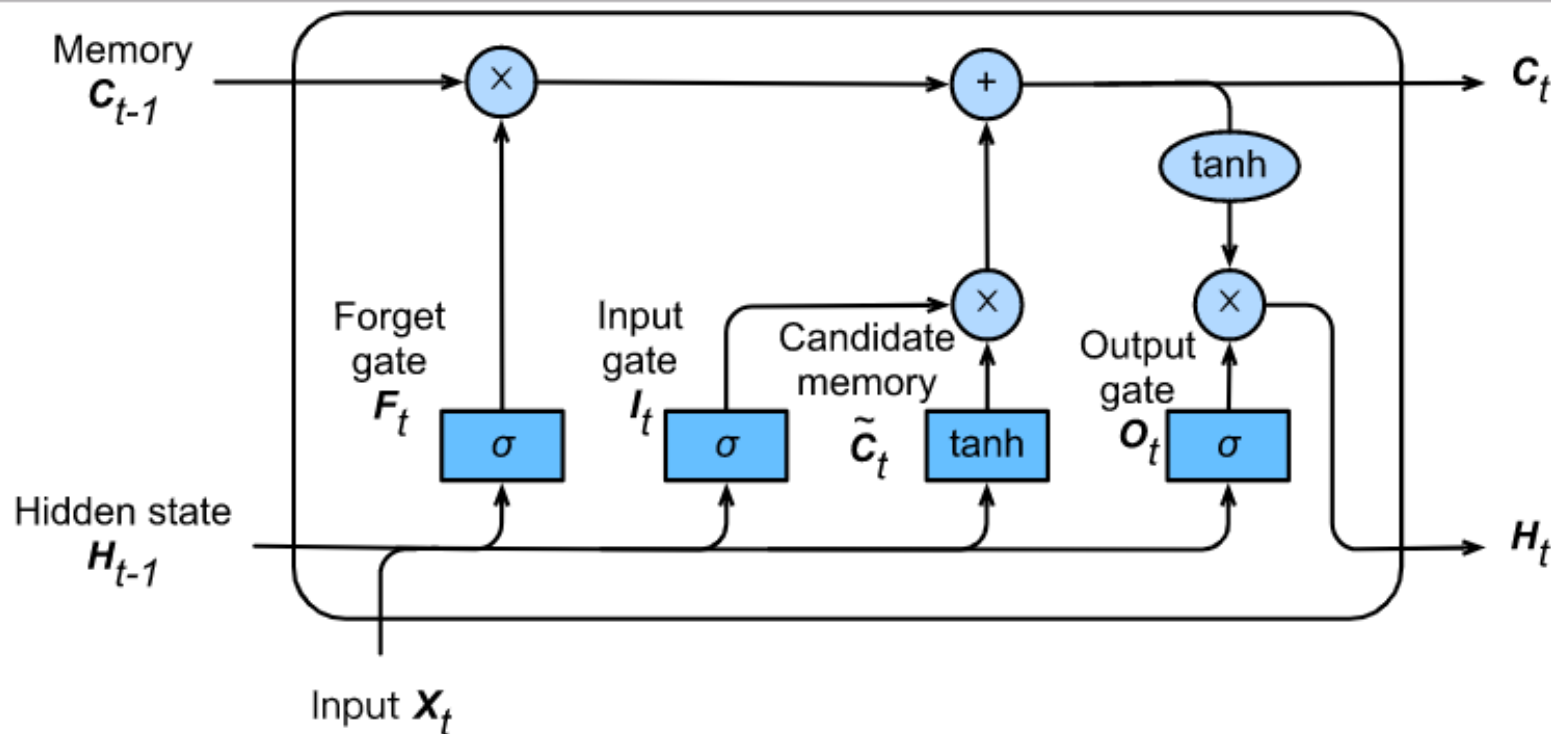


Concatenate

Fonte: https://www.d2l.ai/chapter_recurrent-neural-networks/lstm.html

Long Short-Term Memory (LSTM)

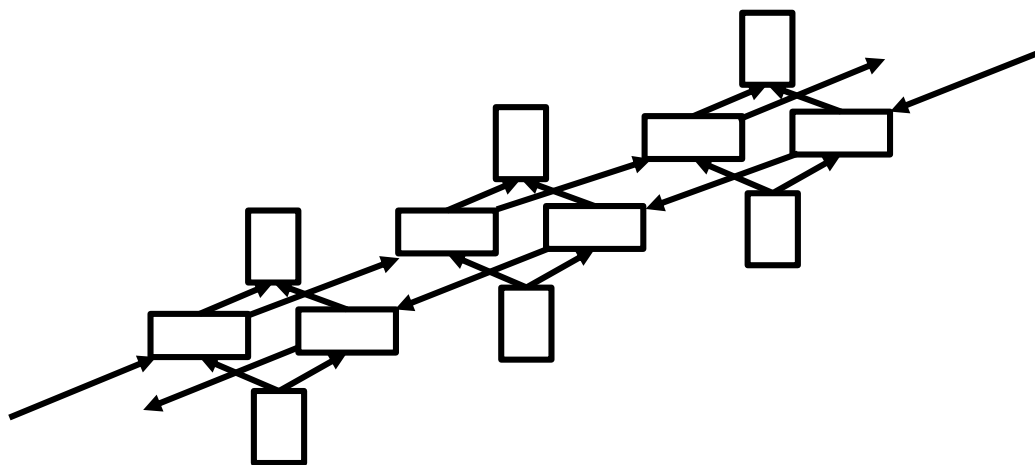
► Estado escondido



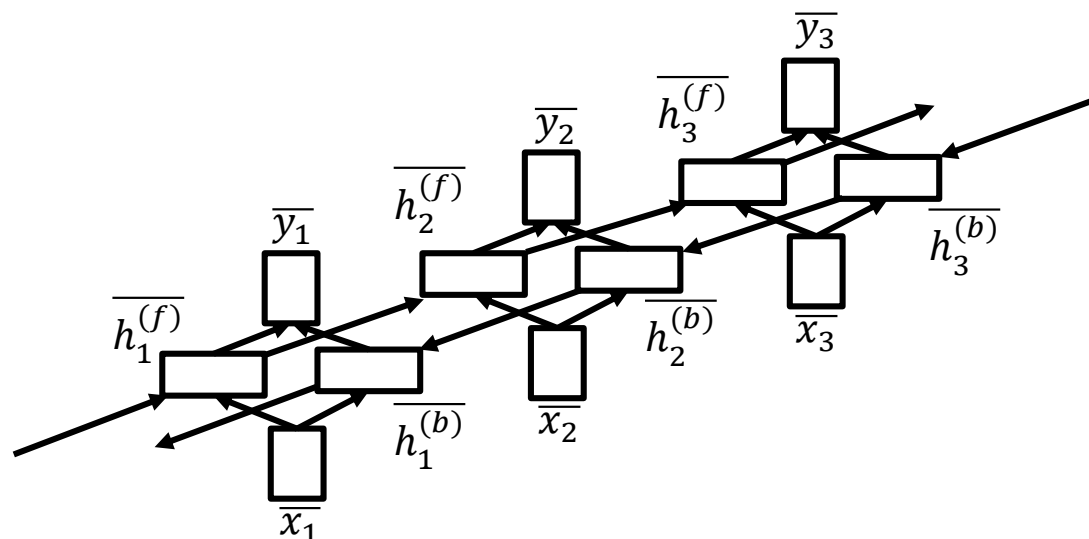
Fonte: https://www.d2l.ai/chapter_recurrent-neural-networks/lstm.html

Bidirectional Recurrent Networks (BRNN)

- ▶ Uma desvantagem de redes recorrentes é que um estado em um determinado intervalo de tempo só tem conhecimento das entradas passadas
 - ▶ Exemplo: “Maria e João entraram na sala. _____ saiu. João ficou.”
- ▶ Redes bidirecionais tentam sobrelevar este problema

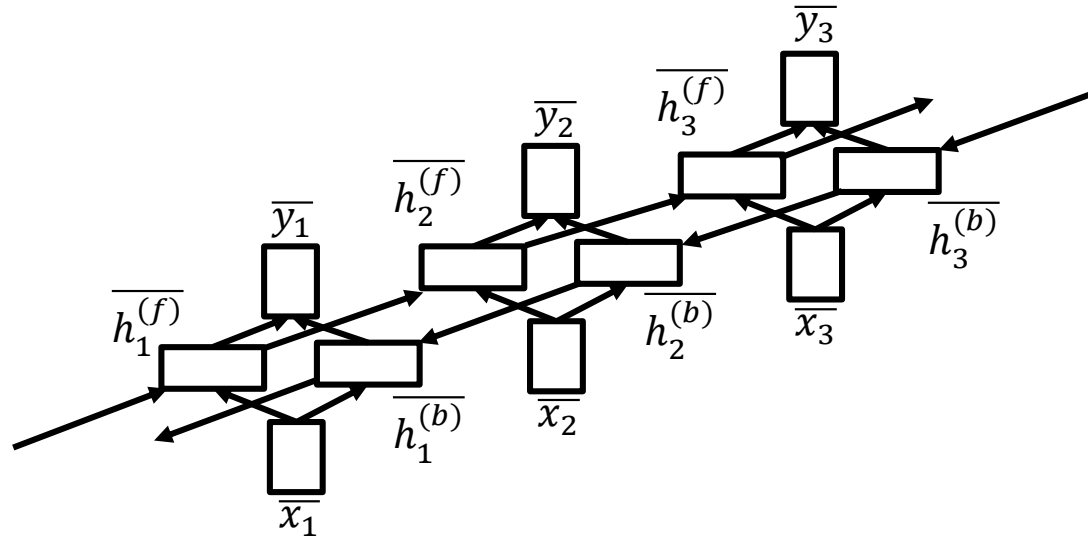


Bidirectional Recurrent Networks (BRNN)



- ▶ Há estados escondidos separados para a fase de ida $\bar{h}_t^{(f)}$ e para a fase de volta $\bar{h}_t^{(b)}$
- ▶ As matrizes de pesos, também, são separadas $W_{xh}^{(f)}$, $W_{hh}^{(f)}$, $W_{hy}^{(f)}$, $W_{xh}^{(b)}$, $W_{hh}^{(b)}$, $W_{hy}^{(b)}$

Bidirectional Recurrent Networks (BRNN)



$$\bar{h}_t^{(f)} = \tanh(W_{xh}^{(f)} \bar{x}_t + W_{hh}^{(f)} \bar{h}_{t-1}^{(f)})$$

$$\bar{h}_t^{(b)} = \tanh(W_{xh}^{(b)} \bar{x}_t + W_{hh}^{(b)} \bar{h}_{t+1}^{(b)})$$

$$\bar{y}_t = W_{hy}^{(f)} \bar{h}_t^{(f)} + W_{hy}^{(b)} \bar{h}_t^{(b)}$$